

# ELI — Background Tasks

ELI v2.1.65

August 2026

## Contents

|                                                                        |          |
|------------------------------------------------------------------------|----------|
| <b>ELI Background Tasks &amp; Unified Code Generation</b>              | <b>1</b> |
| 1. Unified code generation . . . . .                                   | 1        |
| 2. Background task manager (eli/runtime/background_tasks.py) . . . . . | 2        |
| Control summary . . . . .                                              | 2        |
| Scope / honesty . . . . .                                              | 2        |

## ELI Background Tasks & Unified Code Generation

Two related additions: 1. **All code generation now routes through the verified coding agent** — asking for a script, or self-upgrading, runs the plan → DAG/tree-search → execute → repair → bug-memory pipeline instead of a single LLM shot. 2. **Heavy work runs on background threads** — when a task is estimated heavy (or you ask), ELI starts it on a worker thread, hands back a job id immediately, and you check on it later.

### 1. Unified code generation

| Path                                                   | Before                                                           | Now                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CODE_SOLVE<br>GENERATE_SCRIPT                          | the coding agent<br>inline single-shot prompt +<br>static checks | the coding agent (unchanged)<br><b>routes through<br/>eli.coding.solve first</b><br>(plan/DAG/tree-<br>search/verify/repair), saving a<br>verified, top-tier script; falls<br>back to the inline generator only<br>if the agent yields nothing or<br>ELI_GENERATE_SCRIPT_AGENT=0<br>now also <b>consults the coding<br/>engine’s long-term bug<br/>memory</b> : classifies the failure<br>and injects prior fixes for that<br>bug class into the patch prompt |
| Self-upgrade<br>(self_improvement.generate_code_patch) | LLM patch + import-verify                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

So “write a script”, “generate code”, and “improve ELI” all share one engine and one bug-memory — the same consolidation move applied to recall\_memory. Kill switch: ELI\_GENERATE\_SCRIPT\_AGENT=0 restores the legacy inline generator.

## 2. Background task manager (eli/runtime/background\_tasks.py)

In-process, multi-threaded (ThreadPoolExecutor). **Distinct** from eli/planning/jobqueue.py (which runs durable *external subprocess* jobs) — this runs live in-process Python work (agent runs) for the session.

- BackgroundTasks.submit(name, fn, \*args) → integer job id; tracks queued → running → done/failed/cancelled, result, error, elapsed, note.
- get(id), list(), cancel(id) (best-effort), wait(id), stats().
- Singleton get\_background\_tasks(); thread-safe.

### When ELI backgrounds a task (eli/coding/cost.py)

should\_background(task) decides: - **Explicit phrasing wins**: “in the background / don’t wait / notify me when” → background; “right now / quickly / wait for it” → foreground. - **Otherwise heuristic**: estimate\_cost scores heavy-compute signals (simulate/monte-carlo/optimise/benchmark/train/dijkstra/parallel...), multi-component structure, length, and planned-step count. heavy ( $\geq 0.6$ ) = background.

\_maybe\_background\_codegen (executor) applies this at the top of CODE\_SOLVE and GENERATE\_SCRIPT: if heavy/explicit, it submits the same action as a background job (re-dispatched with \_no\_background so the worker doesn’t re-background) and replies “started as job #N — say ‘check job N’.” Kill switch: ELI\_CODEGEN\_BACKGROUND=0.

### Inspecting jobs

- Actions **CHECK\_JOB** ({job\_id} or parsed from “check job 3”) and **BACKGROUND\_JOBS** (list), both in SUPPORTED\_ACTIONS.
- Router triggers: “check job N” / “job N”, “background jobs” / “list jobs”.

### Control summary

| Knob                                                      | Default | Effect                                                      |
|-----------------------------------------------------------|---------|-------------------------------------------------------------|
| ELI_GENERATE_SCRIPT_AGENT                                 | on      | GENERATE_SCRIPT uses the coding agent (off ⇒ legacy inline) |
| ELI_CODEGEN_BACKGROUND                                    | on      | auto-background heavy codegen (off ⇒ always foreground)     |
| ELI_AGENT_DAG / ELI_CODING_DAG                            | on      | DAG dispatch / subtask DAG (see dag.md)                     |
| ELI_CODING_SANDBOX / *_RUN_TIMEOUT / *_BEAM / *_MAX_ITERS | —       | coding-agent execution knobs                                |

### Scope / honesty

- **Tested deterministically** (tests/test\_background.py): manager run/fail reporting, cost light-vs-heavy + explicit phrasing, and the CHECK\_JOB / BACKGROUND\_JOBS executor round-trip. No model needed.
- **Threads, not processes**: background tasks share the process; a running thread can’t be force-killed (cancel is best-effort / cooperative). True OS isolation for a job would use the subprocess jobqueue.
- **“LLM deems longer wait”**: currently a deterministic estimator (+ explicit phrasing). The planner’s decomposition (plan\_steps) can be fed in for an LLM-informed estimate — hook present, not yet auto-wired.
- **GUI**: a dedicated Coding/Jobs tab is not built yet (see note below); jobs are reachable via chat (“check job N”, “background jobs”) and the action API today.